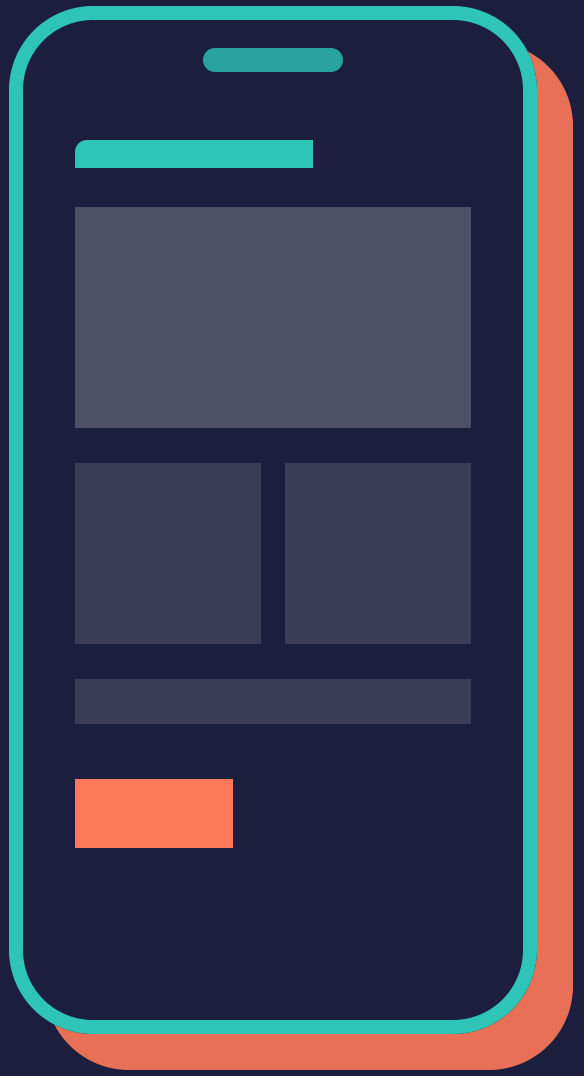
Fondations de React Native

Cours 1 – Architecture, web, composants et responsive

Chris CHEVALIER



Sommaire

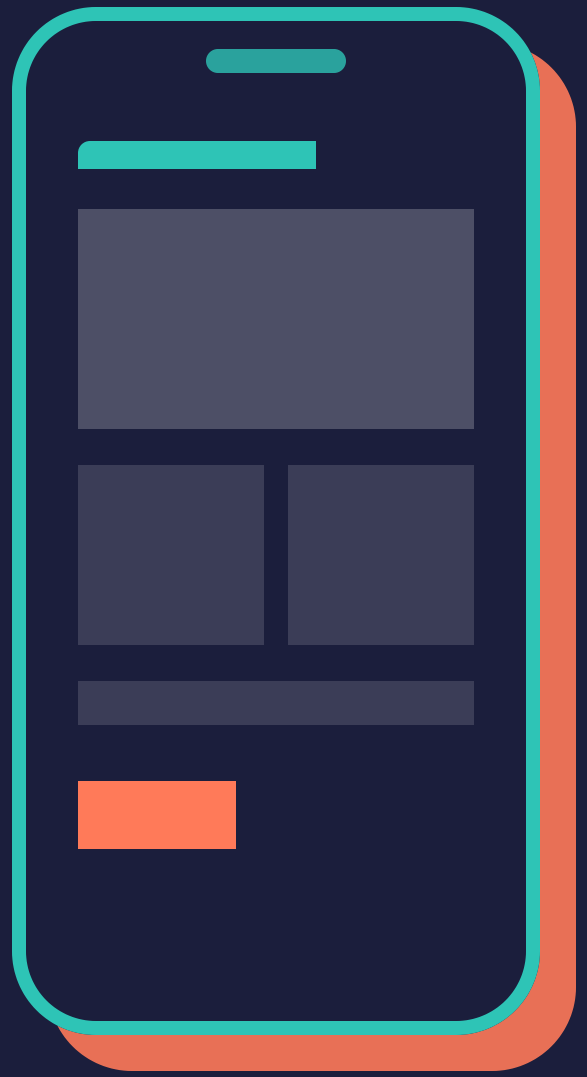
01 De React à React Native

02 Expo, Expo Router et TypeScript

03 Les composants primitifs

04 Styles et Flexbox

05 Une interface responsive



01

De React à React Native

Ce que vous savez déjà, ce qui change



Le même modèle, une autre cible

	React (web)	React Native
Ce que vous écrivez	Des composants, des props, de l'état, des hooks	
Les briques	`<div>`, `<span>`, `<img>`, `<input>`	`<View>`, `<Text>`, `<Image>`, `<TextInput>`
Les styles	CSS, cascade, sélecteurs	Objets JavaScript, aucune cascade
Le rendu	Le DOM du navigateur	Des vues natives iOS et Android (et le DOM sur le web)
La navigation	Des URL	Une pile d'écrans (et des URL sur le web)

> Votre logique React se réutilise telle quelle. Ce qui change, ce sont les **briques d'affichage** et les **contraintes du mobile**.

Comment JavaScript devient une vue native

1. Votre code

Vos composants TypeScript sont exécutés par **Hermes**, le moteur JavaScript optimisé pour le mobile

2. Le rendu

React calcule l'arbre des composants. Le moteur de rendu **Fabric** le traduit en vues natives

3. L'écran

iOS affiche une `UIView`, Android une `android.view.View`, le navigateur une `<div>`

Avant : le bridge

JavaScript et le natif échangeaient des messages **asynchrones** sérialisés en JSON. Le pont devenait un goulot d'étranglement.

Aujourd'hui : la nouvelle architecture

JSI (JavaScript Interface) permet des appels **directs** entre JavaScript et le natif. C'est l'architecture par défaut des projets actuels.

React Native for Web

- React Native for Web réimplémente les composants de React Native avec **React DOM**
- `<View>` devient une `<div>`, `<TextInput>` un `<input>`
- `StyleSheet` génère des **classes CSS** optimisées
- Un seul code pour trois plateformes

Ce que vous écrivez

```
<View style={styles.card}>
  <Text>Studio lumineux</Text>
</View>
```

Ce que le navigateur reçoit

```
<div class="css-view-g5y9jx r-…">
  <div dir="auto" class="css-text-…">
    Studio lumineux
  </div>
</div>
```

Quiz

Une application React Native peut-elle utiliser `<div>` ?

> Non. `<div>` n'existe que dans le DOM : l'application planterait sur iOS et Android. Utilisez `<View>`, que React Native for Web transforme en `<div>` dans le navigateur.

Une application React Native est-elle une page web dans une application ?

> Non. Sur mobile, ce sont de vraies vues natives. Une page web affichée dans une application s'appelle une **WebView** (c'est l'approche de Cordova ou d'Ionic).

02

Expo, Expo Router et TypeScript

Créer et lancer un projet



Expo : le framework recommandé

- **Expo** est le framework que l'équipe de React Native recommande pour démarrer un projet
- Il fournit une CLI (`npx expo`), des modules natifs (caméra, localisation, notifications…) et **Expo Router** pour la navigation
- **Expo Go** : une application à installer sur votre téléphone pour tester votre code sans rien compiler
- Les dossiers natifs `ios/` et `android/` sont générés à la demande. Vous ne les écrivez jamais à la main

> Méfiez-vous des tutoriels anciens et des réponses d'IA. `expo eject` , `expo-cli` installé globalement ou « workflow managed » sont des notions **disparues**.

Créer et lancer le projet

Terminal

```
npx create-expo-app@latest mon-app
cd mon-app
npm run reset-project
npx expo start
```

`reset-project` retire l'exemple fourni et laisse un projet vide.

- **Sur le web** : appuyez sur `w` dans le terminal
- **Sur votre téléphone** : scannez le QR code avec Expo Go (Android) ou l'appareil photo (iOS)
- **Sauvegardez un fichier** : l'écran se met à jour aussitôt (Fast Refresh)

> Le Wi-Fi de l'école bloque la connexion du téléphone ? Lancez `npx expo start --tunnel`.

L'arborescence du projet

mon-app/

```
src/
├── app/              écrans et navigation (Expo Router)
│   ├── _layout.tsx   navigateur racine
│   └── index.tsx     écran de l'URL « / »
├── components/       composants réutilisables
├── hooks/            hooks personnalisés
├── models/           types des données
└── data/             données locales
assets/               images, polices
app.json              configuration Expo
tsconfig.json         configuration TypeScript
```

- **Chaque fichier de `src/app/` est un écran**. Son chemin donne son URL
- Tout le reste du code vit **en dehors** de `src/app/`
- L'alias `@/` pointe vers `src/` : `import { STAYS } from "@/data/stays"`

Un réglage pour le web : `web.output`

- Le modèle actuel d'Expo rend les pages **sur un serveur** (`"server"`) avant de les envoyer au navigateur
- Le serveur ne connaît pas la taille de la fenêtre. Un écran responsive y est calculé différemment : React signale alors une erreur d'**hydratation**
- Dans ce module, on produit une **application** (SPA) : réglez `"single"`

app.json (extrait)

```
{
  "expo": {
    "web": {
      "output": "single",
      "favicon": "./assets/images/favicon.png"
    }
  }
}
```

> **Hydratation** : React rattache ses événements au HTML reçu du serveur. Si le HTML ne correspond pas au premier rendu dans le navigateur, l'hydratation échoue.

TypeScript : ce qui nous servira

src/models/stay.ts

```ts
export type StayType = "apartment" | "house" | "cabin";

export type Stay = {
  id: string;
  title: string;
  city: string;
  type: StayType;
  pricePerNight: number;
  available: boolean;
  imageUrl?: string;
};
```

- `type` décrit la **forme** d'une donnée
- `"apartment" | "house"` : une **union**, seules ces valeurs sont acceptées
- `imageUrl?` : propriété **facultative** (`string | undefined`)
- TypeScript vérifie dans l'éditeur et avec `npx tsc --noEmit` , **avant** l'exécution

Des données typées

src/data/stays.ts

```ts
import { Stay } from "@/models/stay";

export const STAYS: Stay[] = [
  {
    id: "s1", title: "Studio lumineux", city: "Paris",
    type: "apartment", pricePerNight: 95, available: true,
  },
  {
    id: "s9", title: "Yourte", city: "Nantes",
    type: "tent", pricePerNight: 70, available: true,
    // ↑ erreur : "tent" n'est pas un StayType
  },
];
```

> Une faute de frappe ou un champ oublié est signalé **dans l'éditeur**, avant même de lancer l'application.

03

Les composants primitifs

Les briques de toute interface



Les briques de base

Composant	Rôle	Équivalent web
`View`	Conteneur et mise en page	`<div>`
`Text`	Tout texte affiché	`<p>`, `<span>`
`Image`	Image locale ou distante	`<img>`
`TextInput`	Saisie de texte	`<input>`
`Pressable`	Zone qui réagit au toucher	`<button>`
`ScrollView`	Contenu qui défile (court)	`overflow: auto`
`FlatList`	Liste longue, affichée au fur et à mesure	Aucun

Tous s'importent depuis `react-native` : `import { View, Text } from "react-native";`

`View` et `Text`

greeting.tsx

```tsx
type GreetingProps = { firstName: string; unread?: number };

function Greeting({ firstName, unread = 0 }: GreetingProps) {
  return (
    <Text style={styles.greeting}>
      Bonjour <Text style={styles.bold}>{firstName}</Text>
      {unread > 0 && ` · ${unread} message(s)`}
    </Text>
  );
}

<Greeting firstName="Manon" unread={3} />
```

- **Tout texte doit être dans un `<Text>`**. Un texte posé dans une `<View>` fait planter l'application
- Un `<Text>` peut contenir d'autres `<Text>`. Ce sont les seuls à hériter du style de leur parent
- Les props sont typées : `<Greeting />` sans `firstName` est refusé par TypeScript

Image

Deux sources possibles

```tsx
// Image locale : intégrée à l'application
<Image
  source={require("@/assets/images/icon.png")}
  style={{ width: 64, height: 64 }}
/>

// Image distante : téléchargée à l'affichage
<Image
  source={{ uri: stay.imageUrl }}
  style={{ width: "100%", aspectRatio: 4 / 3 }}
/>
```

- Une image distante n'a **pas de taille par défaut**. Sans largeur ni hauteur, elle reste invisible
- `aspectRatio` fixe la hauteur à partir de la largeur
- Pour des besoins avancés (cache, transitions), Expo propose `expo-image`

`TextInput` : un champ contrôlé

name-form.tsx

```tsx
function NameForm() {
  const [name, setName] = useState("");
  return (
    <View style={styles.row}>
      <TextInput
        value={name}
        onChangeText={setName}
        placeholder="Votre prénom"
        style={styles.input}
      />
      <Text>{name ? `Bonjour ${name} !` : "…"}</Text>
    </View>
  );
}
```

- Même principe qu'en React : l'état est la **source de vérité**
- `onChangeText` reçoit directement le texte, sans objet événement
- Props utiles : `keyboardType`, `secureTextEntry`, `autoCapitalize`

`Pressable` et le cycle de vie

counter.tsx

```tsx
function Counter() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    console.log("Counter affiché");
    return () => console.log("Counter retiré");
  }, []);

  return (
    <Pressable onPress={() => setCount((c) => c + 1)}>
      <Text>Appuyé {count} fois</Text>
    </Pressable>
  );
}
```

- `onPress` remplace `onClick`
- Un composant est **monté**, **mis à jour** à chaque changement d'état, puis **démonté**
- `useEffect(…, [])` s'exécute après le premier affichage. La fonction renvoyée s'exécute au démontage

Un retour visuel au toucher

stay-card.tsx (extrait)

```tsx
<Pressable
  onPress={() => onToggleFavorite(stay.id)}
  style={({ pressed }) => [
    styles.button,
    pressed && styles.buttonPressed,
  ]}
>
  <Text style={styles.buttonLabel}>
    {isFavorite ? "★ Favori" : "☆ Ajouter aux favoris"}
  </Text>
</Pressable>
```

- Sur mobile, il n'y a pas de survol. L'utilisateur doit **voir** que son doigt a été pris en compte
- `style` accepte une fonction qui reçoit `{ pressed }`
- Un tableau de styles applique les suivants par-dessus les premiers. `false` est ignoré

Afficher une liste : `FlatList`

src/app/index.tsx (extrait)

```tsx
<FlatList
  data={visibleStays}
  keyExtractor={(stay) => stay.id}
  renderItem={({ item }) => (
    <StayCard
      stay={item}
      isFavorite={favorites.includes(item.id)}
      onToggleFavorite={toggleFavorite}
    />
  )}
  ListEmptyComponent={<Text>Aucun résultat</Text>}
/>
```

- `FlatList` n'affiche que les éléments **visibles** (virtualisation)
- `keyExtractor` joue le rôle de la prop `key` de `map()`
- `ScrollView` affiche tout d'un coup : réservez-la aux contenus courts

04

Styles et Flexbox

Du CSS sans CSS



StyleSheet

stay-card.tsx (extrait)

```tsx
const styles = StyleSheet.create({
  card: {
    flex: 1,
    backgroundColor: "#fff",
    borderRadius: 12,
    overflow: "hidden",
    margin: 8,
  },
  title: { fontSize: 18, fontWeight: "700" },
  price: { fontSize: 16, color: "#0B7A70" },
});
```

- Des propriétés CSS en **camelCase** : `backgroundColor`, `borderRadius`
- Des nombres **sans unité** : des points indépendants de la densité de l'écran
- **Aucune cascade** ni sélecteur. Chaque composant reçoit son style
- TypeScript vérifie les noms et les valeurs

Flexbox : les différences avec le web

Propriété	Web (CSS)	React Native
`display`	`block` par défaut	`flex` partout, toujours
`flexDirection`	`row`	**`column`**
`alignContent`	`normal`	`flex-start`
`flex: 1`	Raccourci de `flex-grow`, `flex-shrink` et `flex-basis`	« Prends toute la place disponible »

Les éléments s'empilent **verticalement** par défaut. Pour les aligner sur une ligne, écrivez `flexDirection: "row"`.

Une hiérarchie en zones

Écran découpé en zones

```
<View style={{ flex: 1 }}>
  <View style={styles.header}>
    <Text style={styles.bold}>Titre</Text>
    <Text>Action</Text>
  </View>
  <View style={{ flex: 1 }}>{/* contenu */}</View>
</View>

header: {
  flexDirection: "row",
  justifyContent: "space-between",
  alignItems: "center",
  padding: 12,
},
```

- `justifyContent` : alignement sur l'**axe principal** (vertical par défaut)
- `alignItems` : alignement sur l'**axe secondaire**
- `gap` espace les enfants sans marges
- Une zone de titre, une zone de contenu, une zone d'actions : l'utilisateur comprend l'écran en quelques secondes

Quiz

Je place trois `<Text>` dans une `<View>` sans style. Comment s'affichent-ils ?

> En colonne, l'un sous l'autre : `flexDirection` vaut `column` par défaut.

Je donne `color: "red"` à une `<View>`. Le texte qu'elle contient devient-il rouge ?

> Non. Il n'y a pas de cascade : seul un `<Text>` parent transmet son style à un `<Text>` enfant.

05

Une interface responsive

Du téléphone à l'écran de bureau



Pas de media queries : un hook

src/hooks/use-columns.ts

```ts
import { useWindowDimensions } from "react-native";

// Nombre de colonnes selon la largeur de la fenêtre
export function useColumns(): number {
  const { width } = useWindowDimensions();
  if (width >= 1024) return 3;
  if (width >= 600) return 2;
  return 1;
}
```

- `StyleSheet` ne connaît pas les media queries du CSS
- `useWindowDimensions()` renvoie la taille de la fenêtre et **déclenche un nouveau rendu** quand elle change (rotation, redimensionnement)
- L'ancienne API `Dimensions.get()` ne se met pas à jour toute seule : préférez le hook

Une grille sur le web, une liste sur mobile

src/app/index.tsx (extrait)

```tsx
export default function CatalogScreen() {
  const columns = useColumns();
  // …
  return (
    <View style={styles.container}>
      <FlatList
        key={columns}
        numColumns={columns}
        data={visibleStays}
        keyExtractor={(stay) => stay.id}
        renderItem={renderStay}
      />
    </View>
  );
}
```

- `numColumns` transforme la liste en grille
- `FlatList` refuse de changer de nombre de colonnes à chaud. Changer sa `key` force un nouveau montage
- Sur le conteneur, `maxWidth: 1100` et `alignSelf: "center"` évitent une ligne de 2 500 px sur un grand écran

Adapter le code à la plateforme

Platform

```ts
import { Platform } from "react-native";

const label = Platform.select({
  ios: "iPhone",
  android: "Android",
  web: "navigateur",
  default: "autre",
});

if (Platform.OS === "web") { /* … */ }
```

- `Platform.OS` vaut `"ios"`, `"android"` ou `"web"`
- Pour un composant très différent, créez deux fichiers : `map.tsx` et `map.web.tsx`. Le bon est choisi à la compilation
- Utilisez-les avec parcimonie. **Le but reste un seul code**

Les zones sûres de l'écran

Écran plein écran

```tsx
import { SafeAreaView } from "react-native-safe-area-context";

export default function PrimitivesScreen() {
  return (
    <SafeAreaView style={{ flex: 1 }}>
      <ScrollView contentContainerStyle={styles.content}>
        {/* … */}
      </ScrollView>
    </SafeAreaView>
  );
}
```

- Encoche, île dynamique, barre de gestes : certaines zones de l'écran ne doivent pas recevoir de contenu
- `SafeAreaView` ajoute les marges nécessaires
- Importez-la depuis `react-native-safe-area-context` (déjà installé par Expo)

Démo : le catalogue de séjours

- Des données typées (`Stay`)
- Un composant `StayCard` aux props typées
- Une recherche avec `TextInput`
- Des favoris avec `Pressable` et `useState`
- Une grille de 1 à 3 colonnes selon la largeur

Je lance l'application sur le web puis sur mon téléphone avec Expo Go. Redimensionnez la fenêtre : la grille s'adapte en direct.

À retenir

- React Native affiche de **vraies vues natives**. React Native for Web les traduit en DOM
- **Expo** crée, lance et teste le projet. **Expo Router** associe chaque fichier de `src/app/` à un écran
- Tout texte est dans un `<Text>`. Les styles sont des objets, sans cascade
- Flexbox est partout, en **colonne** par défaut
- Le responsive passe par `useWindowDimensions()` et des points de rupture
- TypeScript décrit vos données et vos props, et vérifie avant l'exécution

Des questions ?

Chris CHEVALIER
chevalier@chris-freelance.com

